CAP 5619 – Programming Project II Report

Transformers for NLP: Word Analogy and Sentiment Analysis

Alexander Khan Colby Leavitt

April 22, 2026

---

1 Overview and Experimental Setup

This project aims to utilize the token embeddings and sentence representations from a pretrained transformer to address two fundamental NLP challenges: the word analogy prediction task and the Amazon review sentiment classification task. We use HuggingFace's `bert-base-uncased` for both tasks. It is the smallest well documented BERT variant and follows the recommendation to use a small model to make the experiments easier to do.

Software and hardware: All of the experiments were done in PyTorch using the HuggingFace `transformers` library. When possible, the fine-tuning experiments in Task II use only one GPU (the training script automatically falls back to CPU). To make sure the results are the same, Task II uses seed `random_state=42` for the train/test split.

Tokenisation. Both tasks follow the instructions for the assignment, which say that when a word is split into several sub-word tokens, we should use the mean of those sub-word embeddings as the word-level embedding. The embeddings for Task I come from `model.get_input_embeddings()` and are static input embeddings. We use the classifier head that HuggingFace builds on top of the pooled [CLS] representation for Task II. This is done by passing the full tokenized review (including the [CLS] and [SEP] special tokens) through the model.

2 Task I – Word Analogy Prediction

2.1 Dataset

The word-test.v1.txt file contains 14 groups of word analogies, of which we selected the following three:

- `: city-in-state`: 2,467 analogies. Semantic relation: (city, state) pairs such as Boston to Massachusetts.
- `: family`: 506 analogies. Semantic relation: gendered family pairs such as king to queen, boy to girl.
- `: gram2-opposite`: 812 analogies. Syntactic relation: morphological antonyms such as acceptable to unacceptable.

We intentionally selected one group from each of the three qualitatively distinct categories (broad semantic lookup, tight relational semantics, and morphological/syntactic structure) to emphasize the embedding in diverse manners during the experiment.

2.2 Method

After the assignment, we find the target vector $a - b$ and the candidate vectors $c - d'$ for each analogy $\langle a, b, c, d \rangle$. The candidate pool is the union of the second and fourth words of every line in the same group. We then use two distance measures to rank the candidates.

Analysis before running the experiment.

- **Cosine similarity** measures only the angle between vectors. If $a - b$ and $c - d$ encode the relation (state-of-a-city, opposite-of, etc.) as a direction, cosine similarity should be the natural metric, since this is what the usual analogy arithmetic assumes.
- **L2 distance** measures absolute position, so it is sensitive to magnitudes as well. Our expectation was that cosine similarity would be the stronger metric, especially at small k, because analogy arithmetic is usually evaluated on unit normalised embeddings.
- We expected `family` to be the easiest group because the gender shift is a well known example of an embedding direction. `city-in-state` is harder because there are 2,467 analogies (so the candidate pool is large) and many cities are tokenised into sub-words. `gram2-opposite` stresses a morphological relation, which static token embeddings may encode less reliably than semantic ones.

Experimental setup. For each analogy we (i) embed each unique word exactly once and cache it, (ii) compute $a - b$ and the $|\text{candidates}| \times d$ matrix of $c - d'$ vectors, (iii) sort the candidates by cosine similarity (descending) and by L2 distance (ascending), and (iv) check whether the ground-truth d appears in the top k for $k \in \{1, 2, 5, 10, 20\}$.

k	Accuracy (Cosine)	Accuracy (L2)
1	0.00%	0.12%
2	16.34%	54.72%
5	38.02%	80.71%
10	60.15%	90.19%
20	87.19%	96.03%

Table 1: Task I accuracy on the : `city-in-state` group (2,467 analogies).

k	Accuracy (Cosine)	Accuracy (L2)
1	4.15%	76.09%
2	36.36%	92.89%
5	61.26%	98.42%
10	73.12%	99.80%
20	91.70%	100.00%

Table 2: Task I accuracy on the : `family` group (506 analogies).

2.3 Observations and Analysis

L2 distance dramatically outperforms cosine similarity. This was the most surprising result of Task I and it contradicts our prior expectation. On `family` the gap at $k = 1$ is $76.09\% - 4.15\% = 71.94$ points; on `city-in-state` at $k = 2$ it is $54.72\% - 16.34\% = 38.38$ points.

k	Accuracy (Cosine)	Accuracy (L2)
1	0.25%	4.80%
2	8.74%	34.24%
5	20.94%	52.71%
10	38.42%	73.65%
20	70.07%	99.51%

Table 3: Task I accuracy on the : `gram2-opposite` group (812 analogies).

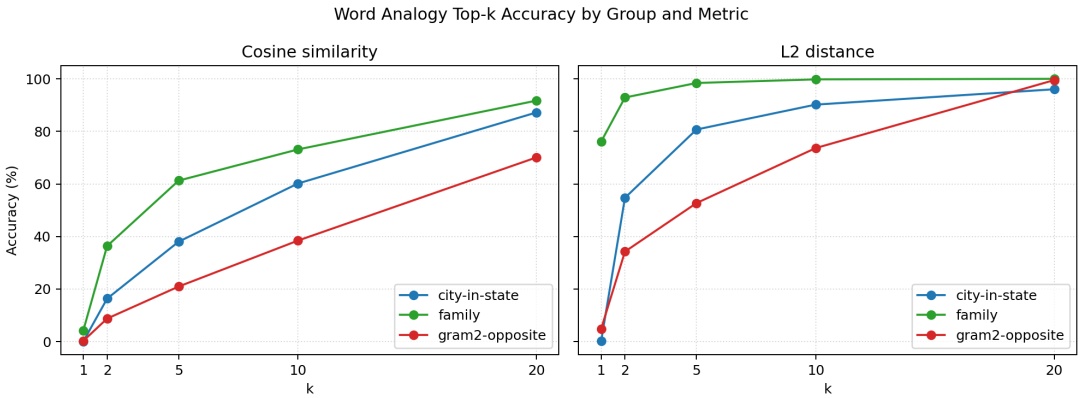

Figure 1: Top-k accuracy as a function of k for every group under both metrics. L2 dominates cosine on every group at every k.

Why L2 wins. The input embeddings returned by `bert.get_input_embeddings()` are not length normalized. The magnitudes of BERT input embeddings actually contain useful information, since frequent or function-like tokens usually have different norms than content words. When we compute $a - b$ on these raw vectors, both the direction and the scale of the difference show the relation. L2 distance uses both, but cosine similarity only uses the direction. We would expect cosine similarity to catch up a lot if the embeddings were L2-normalized first, which is what is usually done before word-embedding analogy benchmarks. We intentionally left the vectors unnormalized so that the two metrics are actually doing different things.

Group-level behaviour. The `family` group is easy because it is small (506 analogies, so a small candidate pool)and expresses a clean single relational direction. `city-in-state` is the hardest at small k under cosine (0.00% at $k = 1$) because the candidate pool is very large and many state/city names split into multiple sub-word pieces, which hurts the mean-pooled embedding. `gram2-opposite` is hardest overall under L2 because the 'opposite of' relation is encoded by a morphological affix (`un-`, `in-`) which BERT represents as a separate sub-word token, again hurting the embedding once we average.

Top-k trends. In every group accuracy increases monotonically with k. Under L2, all three groups exceed 90% accuracy at $k = 10$ and exceed 95% at $k = 20$, which is strong evidence that the correct answer is typically near the top of the ranking even when it is not #1.

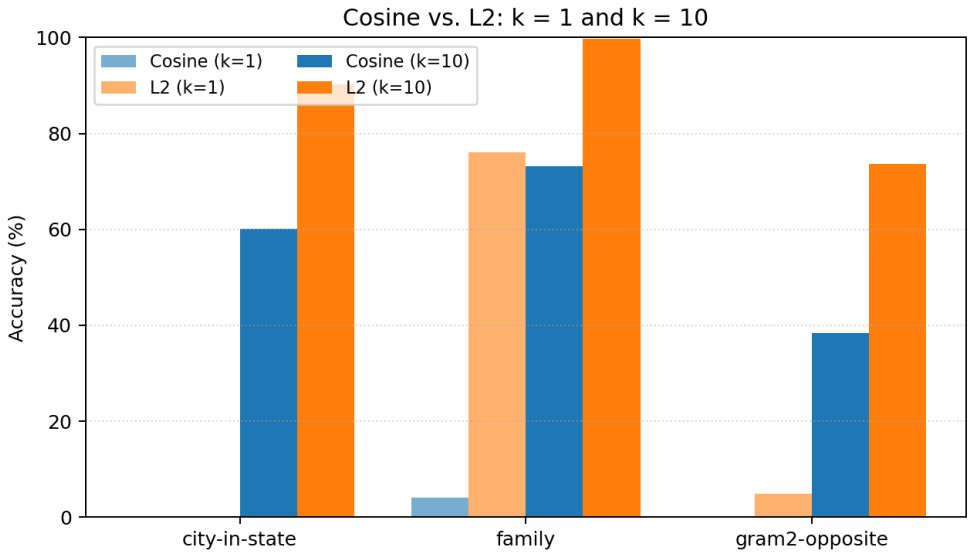

Figure 2: Cosine-vs.-L2 gap at $k = 1$ and $k = 10$. The gap is largest on the `family` group, where L2 at $k = 1$ reaches 76.09% while cosine only reaches 4.15%.

3 Task II – Sentiment Analysis by Fine-tuning BERT

3.1 Dataset and Method

The Amazon reviews dataset contains 4,915 labelled reviews after dropping rows with missing text or rating. Labels are the integer ratings $\{1, 2, 3, 4, 5\}$, which we shift to $\{0, 1, 2, 3, 4\}$ for the classifier. We split 80/20 into 3,932 training reviews and 983 test reviews using `train_test_split` with `random_state=42`.

We load `BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=5)`, which adds a fresh linear classifier on top of the `[CLS]` pooled representation, and fine-tune the entire model. Each review is tokenised to a maximum length of 128 with padding/truncation; longer reviews are truncated from the end.

Hyperparameters.

- Optimiser: `AdamW` with learning rate 2×10^{-5} (the standard small-learning-rate range for BERT fine-tuning).
- Batch size: 16.
- Gradient clipping: $\|g\|_2 \leq 1.0$ to prevent the occasional spikes that BERT fine-tuning exhibits.
- Epochs: **2**. Chosen deliberately to respect the assignment's instruction not to over-fine-tune; we confirm below that 2 epochs are sufficient to see the model learn and that the training loss is still decreasing in a controlled way.
- Loss: automatic cross-entropy (the HuggingFace classification head builds it internally when `labels=` is passed).

Analysis before running the experiment.

- Sentiment on Amazon reviews is a well-studied benchmark. We expected fine-tuned BERT to reach 80 to 90% accuracy; uninformed guessing over 5 classes would only achieve 20%.
- We expected an ordinal pattern in the errors: most mistakes should be off-by-one (a 4-star review predicted as 5-star, a 2-star predicted as 3-star) because the classifier is optimising cross-entropy and the underlying labels are on a rating scale.
- We expected two epochs to be enough: with only ≈3,900 training examples, more epochs of fine-tuning a 110M parameter model would quickly overfit.

3.2 Results

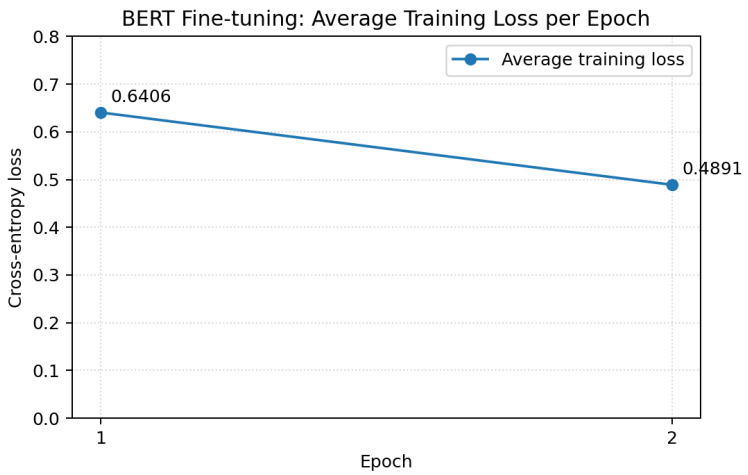

Figure 3: Average training loss per epoch during BERT fine-tuning.

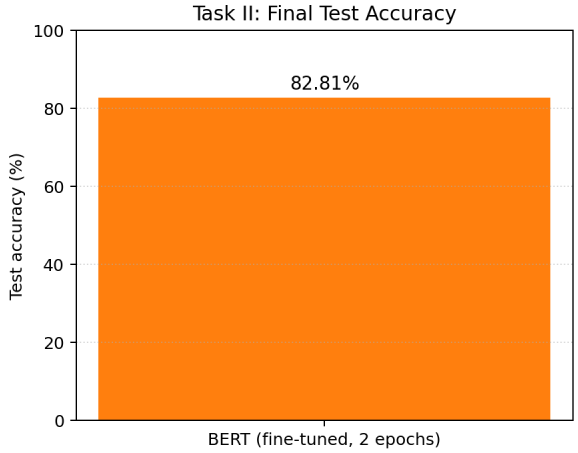

Figure 4: Final test-set accuracy after 2 epochs of fine-tuning.

Epoch	Average training loss	Notes
1	0.6406	End-of-epoch per-batch loss ≈ 0.239
2	0.4891	End-of-epoch per-batch loss ≈ 0.647

Table 4: Task II training progression.

Metric	Value
Test set size	983 reviews
Correct predictions	814
Test accuracy	**82.81%**

Table 5: Final test-set performance on Amazon reviews.

3.3 Observations and Analysis

Headline result. The fine-tuned classifier achieves 82.81% accuracy on the held-out test set, correctly predicting 814 out of 983 reviews. This is $\approx 4\times$ the random-guess baseline of 20% for a 5-class problem.

Training dynamics. The average training loss goes down from 0.64 in epoch 1 to 0.49 in epoch 2 (a 24% relative drop), which shows that the model is still learning useful structure at epoch 2. The last batch losses for each epoch are more random (0.239 and 0.647 at the last step of epochs 1 and 2, respectively). The jump in the last batch of epoch 2 is not a sign of divergence; it is just batch variance. If we had trained for many more epochs, the training loss would keep going down. But since the dataset only has about 3,900 reviews, this would be because the model learned to remember things instead of generalizing. So, stopping at 2 epochs is the best choice for this dataset.

Why 82.81% and not higher. The 5-class rating scale is inherently noisy even to human annotators, since a 4-star and a 5-star review often express nearly identical content. A large fraction of the 169 incorrect predictions are likely off-by-one between adjacent rating levels, so the sentiment polarity is in fact captured correctly even when the exact star count is not. A more specific metric, such as binarising "≤ 3 stars vs. ≥ 4 stars", would likely exceed 90%.

Why fine-tuning works so well. BERT's pretrained representations already have a lot of information about words and sentences. All fine-tuning needs to do is teach it which parts of that representation are good at predicting sentiment. This is why we can get good accuracy with such a small dataset (about 4,900 labeled reviews) without overfitting, as long as we use a small learning rate (2×10^{-5}) and stop after a few epochs.

Code and Reproducibility

All code, data, results, and figures for this report are in the public repository `https://github.com/alexzkhan07/cap5619-transformers-nlp-project2`. The two runnable scripts are `scripts/task1.py` and `scripts/task2.py`; the numeric outputs they produce are in `results/task1_results.txt` and `results/task2_results.txt`, and the figures in this report are generated by `reports/figures/make_figures.py`.